API Security Risk Analysis

Target API: JSONPlaceholder

Testing Tool: Postman

Prepared by: *Ledet Tamiru*

Table of Contents

1 Executive summary....3
2 API Overview....4
3 Scope....4
4 Methodology....5
5 Consolidated Security Findings....6
5.1 Risk 1: Excessive Data Exposure....6
5.2 Risk 2: Unauthenticated Access....7
5.3 Risk 3: Broken Object-Level Access (IDOR)....8
5.4 Risk 4: Missing or Weak Rate Limiting....9
5.5 Risk 5: Insufficient Input Validation....10
6. Risk Severity Summary....12
7. Overall Business Risk Assessment....12
8. Conclusion....13

1 Executive summary

This report presents the results of a basic API security analysis performed on the JSONPlaceholder public REST API. The purpose of this assessment was to analyze the behavior of several API endpoints and identify common security design issues that may affect modern web APIs.

The testing focused on analyzing API responses, authentication behavior, authorization control, and potential data exposure risks.

The assessment was performed using manual HTTP request testing through Postman.

2 API Overview

Item	Description
API Name	Placeholder
API Type	Public REST API
Base URL	https://jsonplaceholder.typicode.com
Purpose	Development and testing environment

JSONPlaceholder is a public demonstration API used by developers to test HTTP requests and simulate backend functionality.

3 Scope

- **APIs Tested:**
  - `/users`, `/users/1`
  - `/posts`, `/posts/1`
  - `/posts?userId=1`
  - `/comments`
  - `/todos`
- **Tools Used:** Postman, Browser DevTools
- **Testing Focus:** Data exposure, authentication & authorization, input validation, rate limiting

Note: JSONPlaceholder is a public test API. This assessment simulates real-world API security evaluation for educational purposes.

4 Methodology

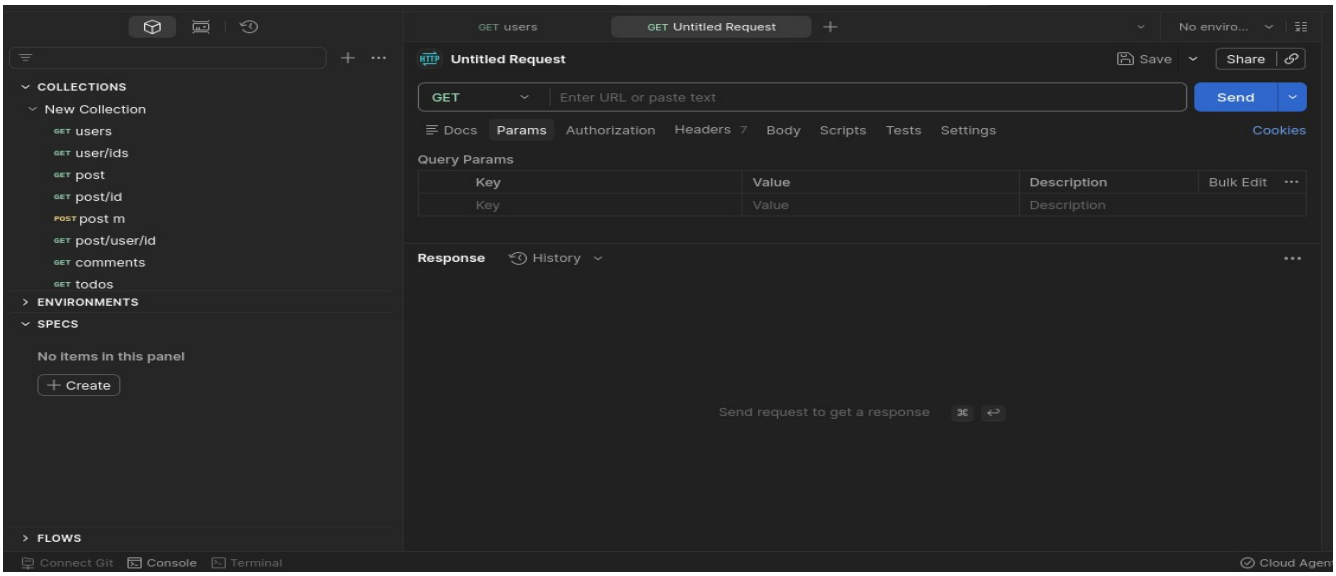
The API testing was performed using a **manual analysis approach**.

The following activities were conducted:

• Sending HTTP GET requests to available API endpoints
• Observing JSON response structures
• Inspecting HTTP status codes
• Testing parameter manipulation
• Evaluating authentication and authorization behavior

The goal was to identify common API security issues such as:

- Excessive data exposure
- Unauthenticated access
- Broken object level authorization
- Weak security controls



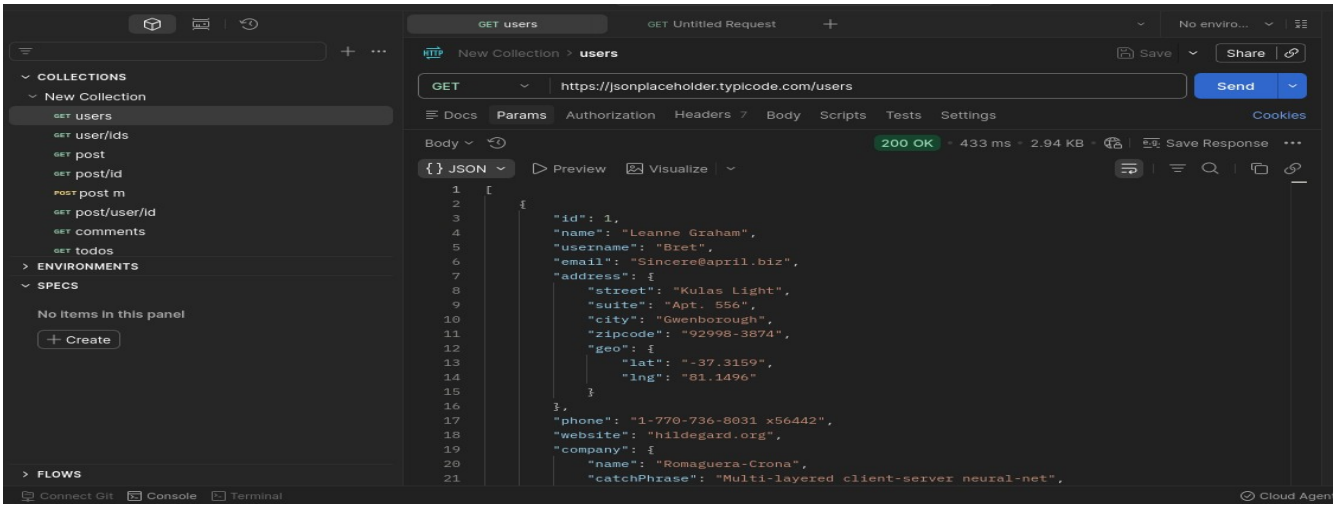
5 Consolidated Security Findings

5.1 Risk 1: Excessive Data Exposure

Affected Endpoints: `/users`, `/users/1`, `/posts`

Description: The API returns complete user and post data without any filtering or restriction. Sensitive fields such as **email addresses, usernames, phone numbers, and address details** are exposed in the response.

Evidence:



The API responses returned full user objects, including:

- Name
- Username
- Email
- Phone number
- Address details

All data was accessible **without authentication**.

Risk Severity: Medium
Business Impact: Public exposure of personal data increases the risk of phishing, spam, or identity misuse.
Remediation Recommendations:

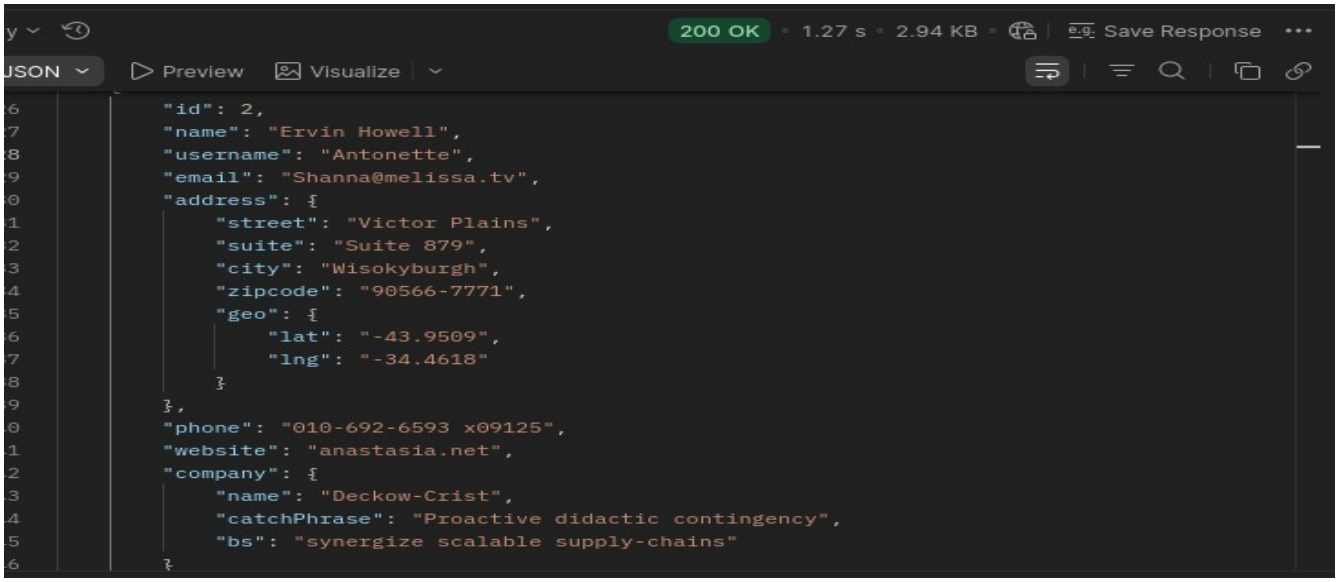
- Implement authentication and authorization
- Limit exposed data to only necessary fields
- Apply data masking for sensitive attributes

5.2 Risk 2: Unauthenticated Access

Affected Endpoints: All tested endpoints (`/users`, `/users/1`, `/posts`, `/comments`, `/todos`)
Description: All endpoints are accessible without authentication. All requests can be executed successfully by any user, which indicates that no authentication or access control mechanisms are enforced. In a real-world application, this would expose sensitive resources to unauthorized users.

Evidence

All data was accessible without any login or token. All requests were executed successfully by an anonymous user.



Risk Severity: Medium
Business Impact: In production, this allows anyone to access all API resources, enabling data breaches or scraping.
Remediation Recommendations:

- Require API keys or token-based authentication (JWT/OAuth)
- Enforce role-based access control for sensitive endpoints
- Restrict access to sensitive endpoints
- Validate and verify tokens on every request

5.3 Risk 3: Broken Object-Level Access (IDOR)

Affected Endpoints: `/users/1`, `/posts/1`, `/posts?userId=1`
Description: Endpoints allow access using predictable sequential IDs with no ownership validation. These endpoints use predictable, sequential IDs to access resources and do not enforce ownership checks or proper authorization. By changing the ID in a request (e.g., `/users/1` → `/users/2`), an attacker can access data belonging to other users. This is known as an **Insecure Direct Object Reference (IDOR)** vulnerability, which can expose sensitive user information.

Evidence:

COLLECTIONS
New Collection
GET users
GET user/ids
GET post
GET post/id
POST post m
GET post/user/id
GET comments
GET todos
ENVIRONMENTS

GET https://jsonplaceholder.typicode.com/users/4 Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Body 200 OK 1.31 s 1.63 KB Save Response

{} JSON Preview Visualize


```
{
    "id": 4,
    "name": "Patricia Lebsack",
    "username": "Karianne",
    "email": "Julianne.OConner@kory.org",
    "address": {
        "street": "Hoeger Mall",
```


COLLECTIONS
New Collection
GET users
GET user/ids
GET post
GET post/id
POST post m
GET post/user/id
GET comments
GET todos
ENVIRONMENTS

GET https://jsonplaceholder.typicode.com/users/3 Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Body 200 OK 7.78 s 1.63 KB Save Response

{} JSON Preview Visualize


```
{
    "id": 3,
    "name": "Clementine Bauch",
    "username": "Samantha",
    "email": "Nathan@yesenia.net",
    "address": {
        "street": "Douglas Extension",
```


COLLECTIONS
New Collection
GET users
GET user/ids
GET post
GET post/id
POST post m
GET post/user/id
GET comments
GET todos
ENVIRONMENTS

GET https://jsonplaceholder.typicode.com/posts/6 Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Body 200 OK 442 ms 1.41 KB Save Response

{} JSON Preview Visualize


```
{
    "userId": 1,
    "id": 6,
    "title": "dolorem eum magni eos aperiam quia",
    "body": "ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid
        molestiae\nperspiciatis et ea nemo ab reprehenderit accusantium quas\nvoluptate
        dolores velit et doloremque molestiae"
```


COLLECTIONS
New Collection
GET users
GET user/ids
GET post
GET post/id
POST post m
GET post/user/id
GET comments
GET todos
ENVIRONMENTS

GET https://jsonplaceholder.typicode.com/posts/7 Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Body 200 OK 470 ms 1.34 KB Save Response

{} JSON Preview Visualize

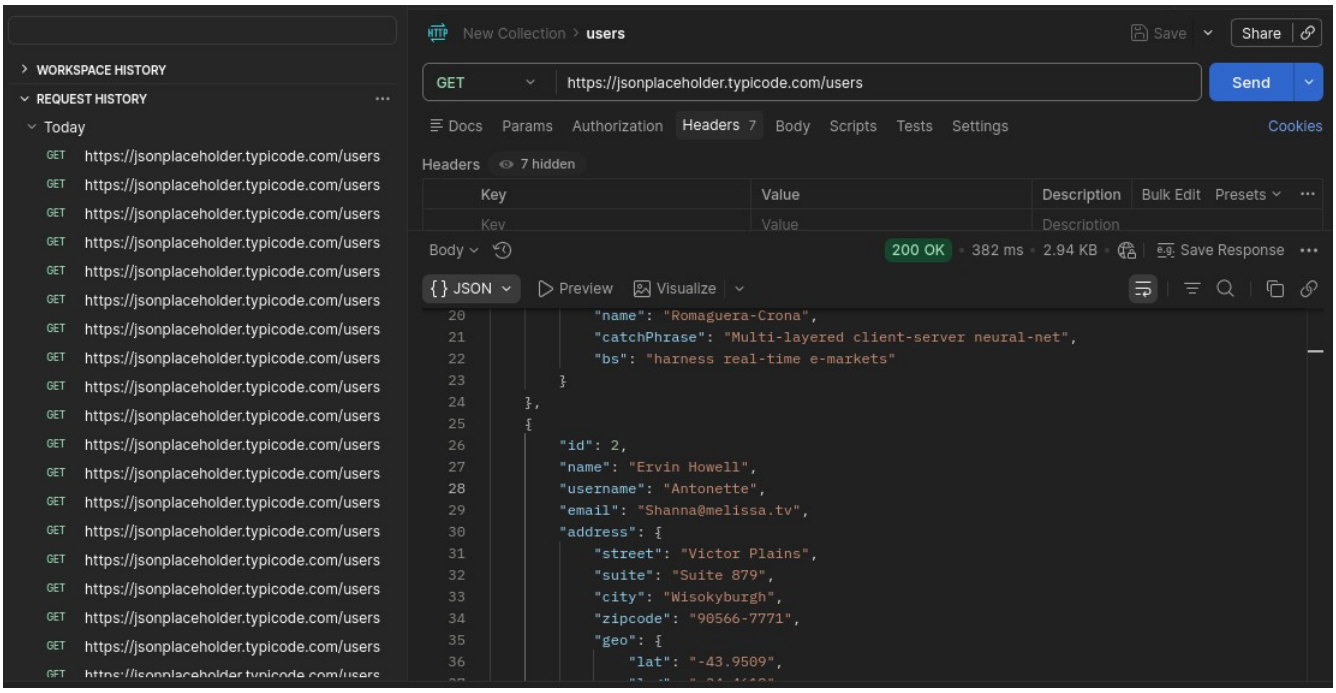

```
{
    "userId": 1,
    "id": 7,
    "title": "magnam facilis autem",
    "body": "dolore placeat quibusdam ea quo vitae\nmagni quis enim qui quis quo nemo aut
        saepe\nquidem repellat excepturi ut quia\nsunt ut sequi eos ea sed quas"
}
```

By changing the ID in a request (e.g., `/users/1` → `/users/2`), an attacker can access data belonging to other users.
Risk Severity: Medium
Business Impact: Attackers could enumerate IDs to access other users' data in production.
Remediation Recommendations:

- Enforce ownership validation for resource access
- Use non-sequential IDs (UUIDs)
- Check user token against requested resource server-side

5.4 Risk 4: Missing or Weak Rate Limiting

Affected Endpoints: All tested endpoints
Description: The API does not enforce any rate-limiting controls, allowing clients to make an unlimited number of requests. This lack of throttling exposes the system to potential abuse by automated tools or attackers.
Evidence:

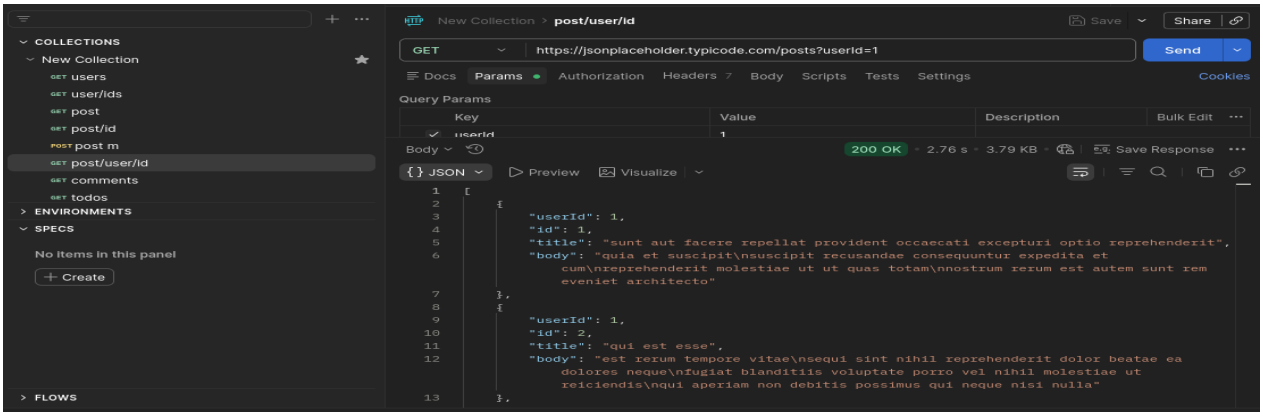


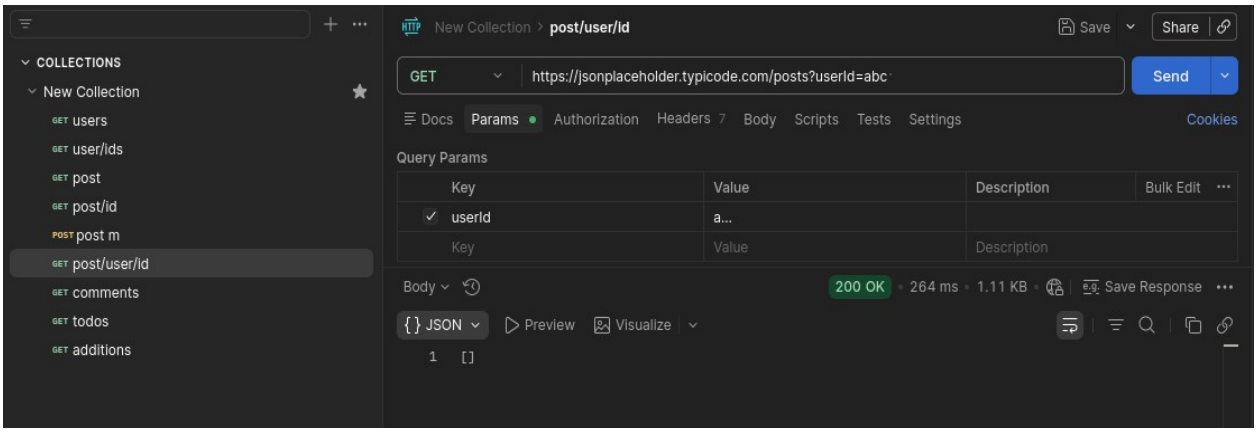
Repeated requests to `/posts` succeeding without restriction, demonstrating lack of rate limiting.
Risk Severity: Low
Business Impact: Without rate limiting, attackers could perform automated scraping of sensitive data or launch Denial-of-Service (DoS) attacks, affecting system availability and performance.
Remediation Recommendations:

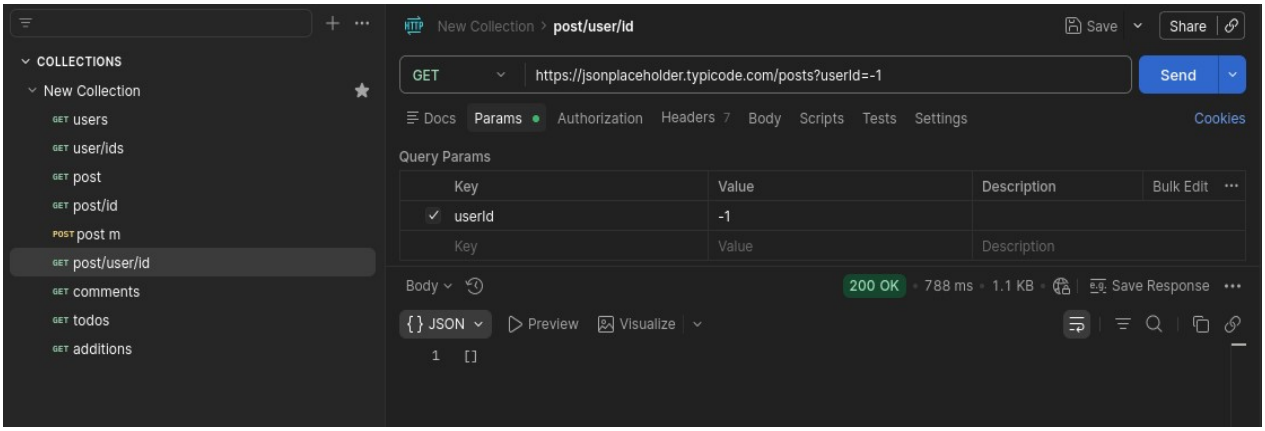
- Apply adaptive rate-limiting
- Monitor and alert on abnormal request patterns

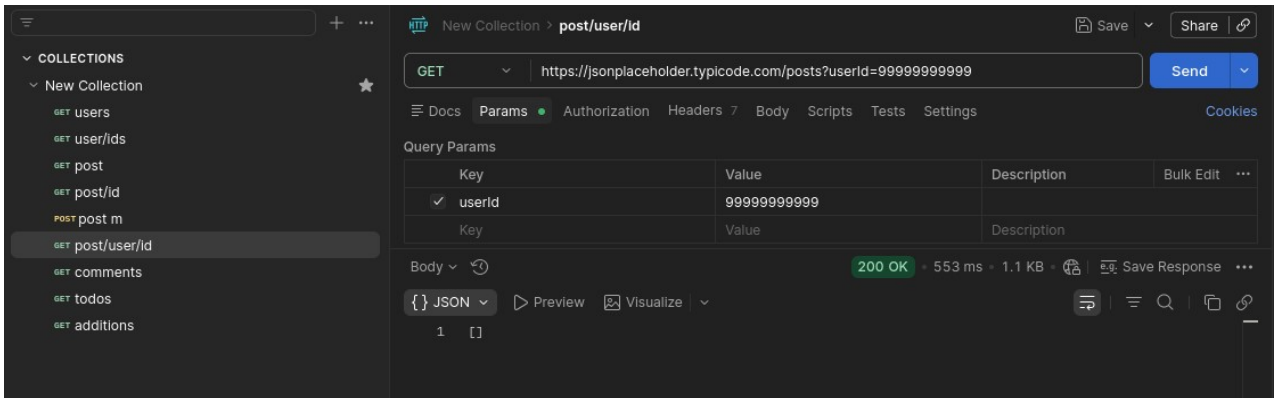
5.5 Risk 5: Insufficient Input Validation

Affected Endpoints: `/posts?userId=1` and other query parameter endpoints
Description: The API does not properly validate query parameters, allowing users to submit unexpected or malformed values. This can lead to application errors or security issues if the input is processed without checks.
Evidence: The system processes input without sufficient checks, leading to application errors if used in production.









Risk Severity: Low
Business Impact: Poor input validation could lead to injection attacks or application errors in production.
Remediation Recommendations:

- Validate all input server-side
- Enforce correct data types and formats

6. Risk Severity Summary

Risk	Severity
Excessive Data Exposure	Medium
Unauthenticated Access	Medium
Broken Object-Level Access (IDOR)	Medium
Missing / Weak Rate Limiting	Low

Insufficient Input Validation	Low

7. Overall Business Risk Assessment

If these issues existed in a production API, the business could face:

- Unauthorized access to user data
- Data scraping and leakage
- Reputation damage and trust loss
- Regulatory compliance issues

8. Conclusion

This analysis shows that API security flaws often result from **design and configuration choices**, not advanced attacks. JSONPlaceholder provides a safe environment to understand common API risks, including **unauthenticated access, excessive data exposure, and IDOR**. Implementing authentication, input validation, rate-limiting, and access control are key mitigation strategies for real-world APIs.